ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Проект: FlightFinder - Веб-сервис для поиска авиабилетов

1. Общие сведения

Название проекта	FlightFinder
Тип проекта	Веб-приложение (Web-сервис)
Основные технологии	Python 3.12, Flask, SQLAlchemy

2. Цель и назначение проекта

Цель: Разработать веб-сервис для поиска и сравнения авиабилетов с возможностью фильтрации по различным параметрам, аналогичный Aviasales, но с уникальным дизайном и дополнительным функционалом.

Задачи, решаемые проектом:

- Предоставление пользователям возможности поиска авиабилетов по любому направлению с актуальными (генерируемыми) ценами и условиями.
- Сохранение понравившихся рейсов в избранное
- Личный кабинет с возможностью загрузки аватара
- История поисковых запросов для авторизованных пользователей
- Предоставление пользователям возможности поиска авиабилетов по любому направлению с актуальными (генерируемыми) ценами и условиями.

3. Функциональные требования

3.1. Обязательный функционал

1. **Главная страница**

- Предоставление пользователям возможности поиска авиабилетов по любому направлению с актуальными (генерируемыми) ценами и условиями.
- Блок с популярными направлениями по России (статические карточки).
- Информация о преимуществах сервиса
- Блок со специальными предложениями (3 самых дешевых рейса из базы данных).
- Навигационная панель с логотипом, ссылками на главную и мои бронирования, а также блоком авторизации.

2. **Поиск и фильтрация (Ключевая особенность)**

- **Динамическая генерация рейсов:** В отличие от классического подхода, поиск не обращается к статической базе данных всех рейсов. Вместо этого, для выбранных пользователем городов и дат система генерирует новые рейсы на лету, используя информацию из базы данных аэропортов (коды, таймзоны) и авиакомпаний.
- **Страница результатов (**/results**):**

- Отображение найденных рейсов (туда и, при выборе, обратно).
- **Фильтрация:**

- По цене (диапазон).
- По количеству пересадок (0, 1, 2+).
- Только рейсы с включенным багажом.
- По времени вылета (утро, день, вечер, ночь).
- **Сортировка:** по возрастанию/убыванию цены, по времени вылета, по длительности.
- **Интерактивность:** При клике на карточку рейса открывается модальное окно с детальной информацией, загружаемой через REST API (/api/flight/<int:flight_id>).
- **Выбор рейсов:** Для маршрута "туда-обратно" реализован двухэтапный выбор: сначала пользователь выбирает билет "туда", после чего для него активируется выбор билета "обратно".

3. **Карточка рейса и бронирование**

- **Страница деталей (**/flight/<flight_id>**):** Отображает полную информацию о рейсе: города, аэропорты, время вылета/прилета в локальном времени, длительность, информацию о пересадках (города и время ожидания), наличие багажа.
- **Форма бронирования:** На странице деталей пользователь может изменить класс обслуживания и количество пассажиров. Цена пересчитывается динамически с помощью JavaScript. Классы обслуживания имеют множители к базовой цене (Эконом 1.0, Комфорт 1.5, Бизнес 2.5, Первый 4.0).
- **Бронирование:** После подтверждения создается запись в БД, генерируется уникальный номер бронирования, и пользователь перенаправляется в личный кабинет.

4. **Регистрация и авторизация**

- Форма регистрации (имя пользователя, email, пароль) с валидацией данных и защитой от повторной регистрации.
- Хеширование паролей с использованием werkzeug.security.

5. **Личный кабинет (**/profile**)**

- Отображение информации о пользователе: имя, email, дата регистрации.
- **Загрузка аватара:** Пункт, закрывающий требование "загрузка и использование файлов". Пользователь может загрузить изображение, которое обрабатывается с помощью Pillow (изменение размера, оптимизация). Аватар отображается в навигационной панели и профиле. Используется url_for для корректной генерации ссылок на загруженные файлы.
- Вкладки:

- **Избранное:** Список рейсов, добавленных пользователем. Можно удалить из избранного.
- **История поиска:** Сохраняет последние 10 запросов пользователя с возможностью очистки истории.

- **Мои бронирования:** Отображает список всех бронирований (как в одну сторону, так и туда-обратно) с краткой информацией и статусом.

6. **REST API**

- /api/airports**:** Эндпоинт для автоподстановки городов. Принимает параметр q и возвращает список городов с их IATA-кодами и названиями аэропортов.
- /api/destinations**:** Аналогичный эндпоинт для автоподстановки направлений, с фильтрацией по выбранному городу отправления.
- /api/flight/<int:flight_id>**:** Эндпоинт для получения детальной информации о конкретном рейсе, включая рассчитанную цену с учетом класса и багажа. Используется в модальных окнах.
- /api/user_favorites**:** Эндпоинт для получения списка ID рейсов, которые пользователь добавил в избранное.

7. **Бронирование туда-обратно**

- Реализована модель RoundTripBooking для хранения бронирований, включающих два рейса (туда и обратно).
- Пользовательский путь: поиск -> выбор билета туда -> выбор билета обратно -> подтверждение -> создание бронирования.

3.2. Дополнительный функционал

- **Избранное** — сохранение понравившихся рейсов в отдельный список
- **История поиска** — сохранение последних запросов пользователя
- **Анимации** — плавные переходы при наведении на карточки рейсов
- **Адаптивный дизайн** — корректное отображение на мобильных устройствах
- Безопасная отмена бронирования: Для отмены бронирования пользователь должен пройти простую CAPTCHA (решить пример "2+2"), что является дополнительной мерой защиты от случайных действий.

4. Нефункциональные требования

4.1. Технологический стек

Компонент	**Технология**	**Обоснование**
Язык программирования	Python 3.12	Изучен в рамках курса
Веб-фреймворк	Flask 3.1.3	Легковесный, гибкий, изучен на занятиях
ORM	Flask-SQLAlchemy 3.1.1	Для работы с БД (требование проекта)
Формы	Flask-WTF 1.2.2	Валидация и защита форм
Аутентификация	Flask-Login 0.6.3	Управление сессиями пользователей
REST API	Flask-RESTful 0.3.10	Создание API (требование проекта)
Работа с изображениями	Pillow 12.1.1	Обработка аватарок
База данных	SQLite	Хранение данных (требование проекта)
Работа с таймзонами	pytz 2026.1.post1	Корректный расчет времени вылета/прилета

4.2. Требования к коду

1. **Чистота кода:**

- Соблюдение PEP 8
- Осмысленные имена переменных и функций
- Отсутствие копипаста (DRY принцип)

2. **Качество проектирования:**

- Модульная структура (разделение на blueprint'ы)
- Использование классов (модели, формы)
- Паттерн Factory для создания приложения
- Разделение логики и представления

3. **Безопасность:**

- Хеширование паролей (werkzeug.security)
- Защита от CSRF (Flask-WTF)
- Валидация загружаемых файлов
- Ограничение размера загружаемых файлов
- Использование secure_filename при сохранении файлов.

4.3. Требования к интерфейсу

- **Цветовая схема:** светлая тема с акцентами синего цвета (#4361ee)
- **Шрифты:** Inter, -apple-system, sans-serif
- **Адаптивность:** корректное отображение на всех устройствах
- **Интуитивность:** понятный интерфейс, не требующий обучения

5. Структура базы данных

5.1. Модели данных

```sql
-- Таблица пользователей
CREATE TABLE users (
    id INTEGER PRIMARY KEY,
    username VARCHAR(80) UNIQUE NOT NULL,
    email VARCHAR(120) UNIQUE NOT NULL,
    password_hash VARCHAR(256) NOT NULL,
    avatar VARCHAR(256) DEFAULT 'default_avatar.png',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Таблица рейсов
CREATE TABLE flights (
    id INTEGER PRIMARY KEY,
    origin VARCHAR(100) NOT NULL,
    destination VARCHAR(100) NOT NULL,
    origin_code VARCHAR(3) NOT NULL,
    destination_code VARCHAR(3) NOT NULL,
    departure_time DATETIME NOT NULL,
    arrival_time DATETIME NOT NULL,
    price INTEGER NOT NULL,
    airline VARCHAR(100) NOT NULL,
    baggage BOOLEAN DEFAULT 0,
    transfers INTEGER DEFAULT 0,
    flight_number VARCHAR(20) UNIQUE,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```sql
-- Таблица избранного
CREATE TABLE favorites (
    id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    flight_id INTEGER NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (flight_id) REFERENCES flights(id),
    UNIQUE(user_id, flight_id)
);

-- Таблица истории поиска
CREATE TABLE search_history (
    id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    origin VARCHAR(100) NOT NULL,
    destination VARCHAR(100) NOT NULL,
    date_from DATE,
    date_to DATE,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

6. Структура проекта

6.1. Корневая директория

- main.py - Главный файл приложения. Содержит весь код: конфигурацию, модели, формы, маршруты, API, вспомогательные функции. Точка входа для запуска.
- requirements.txt - Список всех зависимостей проекта.
- .gitignore - Исключает из Git виртуальное окружение, БД, загруженные файлы.

6.2. Папки в корне

5.2. Папки в корне

- templates/ - Папка с HTML шаблонами (17 файлов).
  - base.html - Базовый шаблон (навигация, футер, flash-сообщения, подключение Bootstrap и JS).
  - index.html - Главная страница.
  - search.html - Расширенная страница поиска.
  - results.html - Страница результатов поиска (вся логика выбора билетов и фильтрации).
  - flight_detail.html - Детальная информация о рейсе.
  - confirm_booking.html - Страница подтверждения бронирования (туда-обратно).
  - profile.html - Личный кабинет.
  - my_bookings.html - Страница со всеми бронированиями пользователя.
  - select_return.html - Страница выбора обратного рейса.
  - login.html, register.html - Страницы аутентификации.
  - cancel_booking_confirm.html, cancel_booking_confirm_roundtrip.html - Страницы подтверждения отмены с CAPTCHA.
  - 404.html, 500.html - Страницы ошибок.
  - round_trip_bookings.html
- static/ - Папка со статическими файлами.
  - uploads/ - Папка для загруженных аватарок пользователей.
  - default_avatar.png - Аватар по умолчанию.

7. Тестовые данные

Для демонстрации работы сервиса создаются следующие тестовые данные:

- **Аэропорты:**

  База данных содержит 70+ аэропортов по всей России с указанием IATA-кодов, названий, городов и часовых поясов.

- **Генерация рейсов:**

  При первом запуске создается 400 статических рейсов для демонстрации на главной странице. При поиске же рейсы **генерируются на лету** для указанных дат и направлений, используя данные об аэропортах и авиакомпаниях. Количество генерируемых рейсов варьируется от 8 до 15 на направление.

- **Авиакомпании:**

10 компаний (Аэрофлот, S7, Уральские авиалинии и др.).

- **Цены:**

  от 3 000 до 80 000 рублей (базовая цена).

- **Пересадки:**

  Распределение вероятностей: 55% без пересадок, 30% с 1 пересадкой, 15% с 2 пересадками. Для рейсов с пересадками генерируется детальная информация (город, аэропорт, время ожидания).

8. Возможности для развития

1. Подключение реального API авиакомпаний
2. Добавление онлайн-оплаты
3. Система отзывов о рейсах
4. Мобильное приложение
5. Уведомления о снижении цен
6. Интеграция с картами (показать на карте)
7. Система рекомендаций на основе истории поиска

9. Список использованных источников

- Документация Flask: https://flask.palletsprojects.com/
- Документация SQLAlchemy: https://www.sqlalchemy.org/
- Материалы курса Яндекс.Лицея

10. Заключение

Проект FlightFinder представляет собой полноценное веб-приложение, реализующее все требования учебной программы. В проекте использованы все изученные технологии, применены современные подходы к разработке, обеспечена безопасность и удобство использования.

Согласовано:

Разработчик: Вардумян Эрик Араратович
Дата: «24» марта 2026г.
Преподаватель: Грачёв Николай Романович